



UNIVERSITY OF
LEICESTER

School of Computing and Mathematical Sciences

CO7201 Individual Project

Preliminary Report

Actor's line learning tool

Anshul Kumar

ak1108@student.le.ac.uk

249054534

Project Supervisor: Benjamin Cham (bc290)

Principal Marker: Richard Craggs (rc340)

Word Count:

27/02/2026

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Anshul Kumar

Date: 27/02/2026

Contents

1. Aims and Objectives	3
2. Requirements	4
3. Technical Specification.....	4
4. Requirements Evaluation Plan.....	5
5. Background Research and Reading list.....	6
6. Time-plan and Risk Plan	9
7. References.....	12

1. Aims and Objectives

1.1 Project Motivation and Relevance

Memorising and rehearsing dialogue is a core part of acting. Traditional rehearsal often requires multiple actors to be physically present, limiting practice due to scheduling constraints and availability. Actors frequently rely on repetition or recordings, which provide limited interaction and structured feedback.

Advances in Artificial Intelligence (AI), Speech Recognition, and Natural Language Processing (NLP) make it possible to simulate interactive dialogue and provide automated feedback. However, few dedicated tools combine dialogue simulation, performance evaluation, and progress tracking within a single mobile application designed for actors.

This project addresses this gap by developing an AI-powered mobile application that enables actors to rehearse scripts independently through dialogue simulation and automated feedback.

Main challenges include:

- Speech-to-Text accuracy across different speaking styles
- Fair and meaningful similarity evaluation
- Real-time responsiveness
- Usable rehearsal workflow design
- Script parsing and role identification

1.2 Overall Aim

To develop and evaluate a mobile application that allows actors to upload scripts, select their role, rehearse dialogue interactively, and receive automated feedback to support memorisation and performance improvement.

1.3 Objectives

- Design structured script management.
- Implement user authentication and data storage.
- Integrate Text-to-Speech (TTS) for non-selected roles.
- Integrate Speech-to-Text (STT) for actor input.
- Implement text similarity algorithms (e.g. Levenshtein Distance).
- Provide similarity scoring and error highlighting.
- Measure response time and accuracy.
- Evaluate usability through structured testing.
- Deliver a cross-platform Flutter prototype.

2. Requirements

2.1 High-Level Requirements

The system should allow users to:

- Practise scripts independently through an AI-assisted rehearsal workflow.
- Interact with simulated dialogue using Text-to-Speech (TTS) playback.
- Speak their lines and receive automated feedback using Speech-to-Text (STT) processing.
- Monitor rehearsal progress using performance tracking and visual feedback.

2.2 Essential Requirements

The application must enable users to:

- **Register and log in** using a secure authentication system.
- **Upload or input scripts** manually for rehearsal sessions.
- **Tag or select character roles** within scripts using structured script parsing.
- **Choose “My Role”** so the system identifies which lines belong to the user.
- **Store scripts** in a structured data format for retrieval and reuse.
- **Listen to other characters’ dialogue** through Text-to-Speech (TTS) integration.
- **Speak their own lines** which are captured using Speech-to-Text (STT) recognition.
- **Compare spoken lines with original script text** using a text similarity algorithm (e.g. Levenshtein Distance).
- **Receive similarity-based feedback** classified as correct, partially correct, or incorrect.
- **View highlighted errors** showing missing or mismatched words.
- **Navigate core application screens**, including login, script upload, rehearsal, and feedback interfaces.

2.3 Desirable Requirements

The system should additionally allow users to:

- Select **different TTS voice options** for characters.
- Adjust **speech playback speed** during rehearsal.
- Practise using **Assisted Mode** (guided rehearsal) or **Challenge Mode** (reduced assistance).
- View **rehearsal history** stored in the system database.
- Track **performance statistics**, such as accuracy percentage and number of attempts.
- Import scripts from external file formats (.txt, .pdf, .docx).
- Measure **response delay timing** between dialogue prompts and user responses.
- View **progress visualisation charts** showing improvement over time.

2.4 Optional Requirements

Future versions may allow users to:

- Receive **semantic similarity feedback** using NLP-based comparison.
- Analyse **emotion or tone** from speech input.
- Detect **filler words** such as “um” or “uh”.
- Use **accent adaptation or pronunciation tuning**.
- Benefit from **custom speech recognition model training**.
- Receive advanced AI-based coaching feedback.
- Store and synchronise data using a **scalable cloud backend**.

3. Technical Specification

The system is designed as a modular, cross-platform mobile application tailored for actors.

3.1 Application Framework: Flutter & Dart

The system is built on the **Flutter** framework using the **Dart** language to achieve a high-performance, cross-platform mobile experience for both Android and iOS. By utilising Flutter's **reactive UI state management**, the application can instantly transition between an AI's speaking turn and an actor's listening turn, ensuring that the interface remains synchronised with the rhythm of the dialogue without the lag typically associated with hybrid mobile frameworks.

3.2 Speech Intelligence: Piper Neural TTS

To simulate a ‘natural-sounding’ scene partner without incurring cloud API costs, the application integrates the **Piper TTS** engine, a fast, local neural text-to-speech system. This technology is utilized because it uses optimized **VITS-based models** to produce human-like stress and intonation patterns locally on the mobile device, which are essential for actors to receive realistic emotional cues during independent practice. The app executes this by bundling pre-trained voice models within the **Data Model Layer**, allowing the AI to generate high-fidelity audio instantly and offline, which effectively eliminates network latency and ensures the “24/7 rehearsal” requirement is met without external dependencies.

3.3 Interactive Feedback: OpenAI Whisper (Local STT)

The system employs a localized version of **OpenAI Whisper** (via the whisper flutter or vosk plugin) to monitor the actor's delivery in real-time. This specific implementation is necessary to provide the immediate, structured feedback required for skill acquisition, as it utilizes a “Tiny” or “Base” quantized model to process audio chunks directly on the smartphone's CPU rather than waiting for cloud transcription. This approach directly addresses the critical need for responsiveness in a live performance context while mitigating the high-impact risk of **Speech Recognition Errors** by allowing the **Service Layer** to run multiple recognition passes locally to ensure the highest possible accuracy for the **Similarity Service**.

3.4 Text Analytics: Regex Parser & Levenshtein Service

The **Service Layer** utilizes a custom **Script Parser** and a **Similarity Service** to bridge the gap between raw text and performance evaluation. **Regular Expressions (Regex)** are used to identify industry-standard script formatting patterns, such as capitalized character names, which allows the system to automatically segment dialogue into interactive roles. Once the actor speaks, the **Levenshtein Distance algorithm** is applied to calculate the “edit distance” between the script and the STT transcription, providing a mathematically grounded accuracy score that is resilient enough to handle minor speech recognition errors while still enforcing strict memorisation standards.

3.5 Infrastructure: SQLite & Cloud Storage

Data persistence is managed through a **Modular Architecture** that optimises both local speed and cloud-based synchronization. **SQLite** is utilised within the local **Data Model Layer** because it provides high-speed, offline access to parsed scripts and performance history, ensuring that rehearsals can continue without interruption even in environments with poor connectivity. To facilitate cross-device access and backup, the system integrates with **AWS S3 (Simple Storage Service)** as a secure backend for storing and synchronizing user script files.

By deploying on the **AWS Free Tier**, the application will utilise **AWS Lambda** for any server-side logic and **Amazon Cognito** for secure user authentication. This transition to AWS ensures the actor’s library remains consistently available for **24/7 rehearsal** while providing a professional, scalable platform to demonstrate the app's deployment capabilities during the final evaluation.

3.6 Resource Optimisation: Dart Isolates

To mitigate the risk of **Performance Latency** during heavy computational tasks like real-time string matching, the app utilizes **Dart Isolates** for multi-threading. This allows the "Performance Module" to calculate complex similarity scores on a background thread, preventing the “User Interface Layer” from freezing and ensuring that the actor’s focus remains entirely on their performance rather than technical navigation.

4. Requirements Evaluation Plan

4.1 Functional Evaluation: Ensuring a Reliable Partner

For an actor to trust an AI partner, the underlying technology must be flawless. We will test the **Service Layer** to ensure that the hand-off between the AI speaking (TTS) and the actor responding (STT) is seamless and synchronised. Central to this is verifying the **Script Parser’s** ability to correctly read a script's layout and assign the right characters to the right roles. Finally, we must ensure that the **Similarity Service**, driven by the Levenshtein algorithm, is actually reflecting the actor’s accuracy, and that all this data is securely saved in the **Data Model Layer** for future review.

4.2 Performance Evaluation: Maintaining the "Flow"

In acting, timing is everything. A “Performance Evaluation” ensures the technology doesn’t get in the way of the art.

- **Responsiveness:** I will measure the “Speech-to-Feedback” time to ensure that the **Performance Module** delivers scores quickly enough that it doesn't break the actor's concentration.
- **Recognition Under Pressure:** Because rehearsals happen everywhere, I need to test how well the STT handles different accents and background noise, directly addressing the risk of recognition errors.
- **Efficiency:** I will monitor how the similarity algorithm runs on mobile hardware to ensure the app remains stable and fast during long, intense rehearsal sessions.

4.3 Usability Evaluation: The Actor’s Experience

This phase looks at the **User Interface Layer** through the eyes of the performer.

- **Intuitive Design:** I want to ensure that navigation is effortless, allowing the actor to go from opening the app to rehearsing their first line in seconds.
- **Meaningful Feedback:** It is vital that the feedback isn’t just a number, but a clear, visual guide that helps the actor see exactly where they missed a line or tripped over a word.
- **Rehearsal Workflow:** I will evaluate the entire loop, listening, speaking, and reviewing, to make sure it feels like a natural, helpful conversation rather than a technical exercise.

4.4 Our Methodology for Improvement

Between April 27 and May 3, I will move from lab testing to real-world application. By bringing in volunteers with actual acting experience, I will use **scenario-based testing** and **usability questionnaires** to see how the app holds up in a real rehearsal context. This human-centered feedback, combined with rigorous **unit and black-box testing**, will allow me to iron out integration issues and refine the system for the final delivery.

5. Background Research and Reading list

Memorising and rehearsing dialogue is central to acting craft, but traditional rehearsal depends heavily on coordinating human scene partners, which restricts practice opportunities and limits structured feedback[10]. Recent work on AI scene partners and script readers shows that actors increasingly use AI-driven tools to practise lines, rehearse scenes, and prepare self-tapes independently, often via mobile devices that support 24/7 rehearsal with responsive, natural-sounding voices[11]. These tools typically combine speech technologies, such as Text-to-Speech (TTS) to play other roles and Speech-to-Text (STT) to monitor the actor’s delivery, with simple performance analytics like line recognition and timing feedback.[1,6,12]

More broadly, research on speech interfaces and accessibility indicates that voice-based interaction can reduce barriers imposed by traditional interfaces, while still facing challenges around recognition accuracy under different accents, speaking styles, and environmental noise[1]. This aligns with your project’s concern for robust STT across varied performances and rehearsal contexts. In parallel, sentence and text similarity research surveys methods ranging from edit distances like Levenshtein to semantic embeddings, highlighting a trade-off between computational simplicity and deeper meaning-level comparison[2]. For a mobile acting app that must respond in real time, lightweight string-based similarity offers an interpretable baseline for scoring line accuracy and highlighting errors, with more advanced semantic methods as future extensions[2].

In human–computer interaction and adaptive learning, studies on intelligent tutoring systems and adaptive mobile interfaces emphasise clear, immediate feedback, low interface complexity, and consistent interaction flows to maintain user engagement and support skill acquisition [3,4]. For acting rehearsal, this translates into an interface where actors can focus on performance rather than navigation, supported by structured script management, clear role selection, and straightforward rehearsal workflows[5]. Mobile performance testing research further stresses the importance of responsiveness, stability, and resource-efficient design when real-time interaction is involved, motivating explicit measurement of response time, recognition accuracy, and usability[7]. Within this landscape, your project is positioned as a focused, actor-centred mobile system that integrates script parsing, AI-driven dialogue simulation (TTS and STT), and lightweight similarity-based feedback to help actors rehearse independently while capturing both technical and user-experience metrics in evaluation.

5.1. Reading List

Research Papers

- D. Kumar and A. Sachan, “Bridging the Gap between Disabled People and New Technology in Interactive Web Application with the Help of Voice,” in *2014 International Conference on Advances in Engineering & Technology Research (ICAETR)*, Unnao, India, 2014, pp. 1-5, doi: 10.1109/ICAETR.2014.7012885.
- T. N. Raju, P. A. Rahana, R. Moncy, S. Ajay, and S. K. Nambiar, “Sentence Similarity - A State of Art Approaches,” in *2023 7th International Conference on Computing Methodologies and Communication (ICCMC)*, Erode, India, 2023.
- Improving the user experience using an intelligent Adaptive User Interface in mobile applications,” in *IEEE Proceedings*, 2022.
- P. Shashanka, G. Srishanth, G. Anitha, G. H. Vardhan, and G. M. Krishna, “Smart Tutoring and Adaptive Learning - Leveraging AI to Personalize Education in Real-Time: A Model for Engaging, Individualized, and Effective Learning Experiences,” in *IEEE Xplore*, 2023.

6. Time-plan and Risk Plan

6.1 Project Time-plan

The project runs from **16 February to 15 May**, aligned with supervision meetings and deliverables. Development follows an incremental approach: design - core implementation - integration - evaluation - final documentation.

6.2 Schedule

Phase	Dates	Duration	Activities
Planning & Research	16-22 Feb	1 week	Scope definition, literature review
Preliminary Report	23 Feb-1 Mar	1 week	Requirements and architecture planning
System Design	2-15 Mar	2 weeks	Architecture and UI design
Core Development 1	16-29 Mar	2 weeks	Login, script upload, parsing
Core Development 2	30 Mar-12 Apr	2 weeks	TTS/STT integration
Similarity & Feedback	13-26 Apr	2 weeks	Scoring and feedback features
Evaluation & Improvements	27 Apr-3 May	1 week	Testing and optimisation
Final Integration	4-10 May	1 week	System stabilisation
Final Documentation	11-15 May	5 days	Final report completion

6.3 Risk Plan

Risk Category	Impact	Error Condition	Specific Technical Fix & Action
Speech Recognition Errors	High	The STT API misinterprets a word, leading to an unfair low score.	Implement a “Phonetic Matching” layer. If the Levenshtein Distance score falls below a certain threshold, the system will re-evaluate the line using a phonetic algorithm (like Soundex) to check if the spoken word sounds like the script word, preventing a technical fail for a minor mispronunciation.
Integration Issues	High	A failure occurs when passing data between the Script Parser and the TTS/STT services.	Execute a Modular Debugging protocol. We will isolate the failing module and run Unit and Black-box tests to identify if the error is in the data format (JSON) or the API handshake. The fix involves standardizing the API interface to ensure data consistency across the Service Layer.
Time Constraints	High	Development of complex features (like scoring) takes longer than the allocated 2-week window.	Initiate Feature De-scoping. If we fall behind during the Similarity & Feedback phase, we will pivot to a “Minimum Viable Product” (MVP) by deploying the simplest version of the scoring algorithm and delaying advanced performance metrics to focus on a stable end-to-end rehearsal loop.
Scope Expansion	Medium	New features are requested that threaten the final documentation deadline.	Implement a Feature Freeze. Any new ideas are logged in a “Future Improvements” backlog but are not coded. This ensures

Risk Category	Impact	Error Condition	Specific Technical Fix & Action
			the Final Documentation phase (11-15 May) remains protected for a high-quality academic submission.
Performance Latency	Medium	The app freezes or lags during the transition between the AI speaking and the user's turn.	Optimization of the Processing Logic. We will move heavy computations, like script parsing and similarity scoring, into Background Threads (Isolates). This keeps the User Interface Layer responsive, ensuring the actor can focus on their performance without distraction.
Limited Testing Participants	Medium	Not enough actors are available for the scheduled evaluation week.	Remote Evaluation Protocol. If in-person testing fails, we will deploy the app via TestFlight (iOS) or Firebase App Distribution and use Usability Questionnaires to gather remote feedback from student volunteers with acting experience.

7 References

- [1] D. Kumar and A. Sachan, "Bridging the Gap between Disabled People and New Technology in Interactive Web Application with the Help of Voice," in *2014 International Conference on Advances in Engineering & Technology Research (ICAETR)*, Unnao, India, 2014, pp. 1-5, doi: 10.1109/ICAETR.2014.7012885.
- [2] T. N. Raju, P. A. Rahana, R. Moncy, S. Ajay, and S. K. Nambiar, "Sentence Similarity - A State of Art Approaches," in *2023 7th International Conference on Computing Methodologies and Communication (ICCMC)*, Erode, India, 2023.
- [3] "Improving the user experience using an intelligent Adaptive User Interface in mobile applications," in *IEEE Proceedings*, 2022.
- [4] P. Shashanka, G. Srishanth, G. Anitha, G. H. Vardhan, and G. M. Krishna, "Smart Tutoring and Adaptive Learning - Leveraging AI to Personalize Education in Real-Time: A Model for Engaging, Individualized, and Effective Learning Experiences," in *IEEE Xplore*, 2023.
- [5] Y. W. Syaifudin *et al.*, "An Implementation of Automatic Dart Code Verification for Mobile Application Programming Learning Assistance System Using Flutter," in *IEEE Proceedings*, 2021.
- [6] G. E. Wong and R. R. Porle, "Real-Time Speech Emotion Recognition Using Deep Learning for Emotion Based Music Recommendation," in *IEEE Proceedings*, 2021.
- [7] H. Kim, B. Choi, and W. E. Wong, "Performance Testing of Mobile Applications at the Unit Test Level," in *IEEE Proceedings*, 2020.
- [8] Flutter, "Flutter Official Documentation," : <https://docs.flutter.dev/>.
- [9] Piper TTS documentation
<https://docs.pipecat.ai/server/services/tts/piper>.
- [10] S. Park *et al.*, "Theatrical Language Processing: Exploring AI-Augmented Improvisational Acting and Scriptwriting with LLMs," *arXiv preprint*, 2025.
- [11] J. Grasset *et al.*, "Virtual Reality Rehearsals for Acting with Visual Effects," in *Proc. Graphics Interface*, 2016.
- [12] G. E. Wong and R. R. Porle, "Speech Emotion Recognition for Performance Interaction," *ResearchGate*, 2026.
- [13] OpenAI Whisper: <https://developers.openai.com/api/docs/guides/speech-to-text/>